Bytecode Circuit

개요

Bytecode Circuit은 스마트 컨트랙트의 바이트코드를 검증하는 회로입니다. 이 회로는 바이트코드가 올바르게 저장되어 있고, 각 바이트가 실제 opcode인지 PUSH 명령어의 데이터인지를 구분하며, 코드 해시가 정확한지 검증합니다.

주요 역할:

- 바이트코드의 구조적 정확성 검증
- PUSH 명령어와 데이터 구분
- 코드 해시 검증 (Keccak256 또는 Poseidon)
- Bytecode Table을 통한 다른 회로와의 데이터 공유
- 바이트코드 언롤링(unrolling)을 통한 선형 표현

파일 위치:

- 메인 파일: `zkevm-circuits/src/bytecode_circuit.rs`
- 회로 구현: `zkevm-circuits/src/bytecode_circuit/circuit.rs`
- 언롤러: `zkevm-circuits/src/bytecode_circuit/bytecode_unroller.rs`

UnrolledBytecode 구조

바이트코드는 회로에서 처리하기 위해 “언롤링(unrolling)” 과정을 거칩니다. 이는 바이트코드를 행(row) 단위로 분해하여 각 바이트에 대한 메타데이터를 추가하는 과정입니다.

BytecodeRow 구조

각 바이트는 다음 정보를 포함하는 행으로 표현됩니다:

```
pub struct BytecodeRow<F: Field> {
pub code_hash: Word, // 바이트코드의 해시
pub tag: F, // Header(0) 또는 Byte(1)
pub index: F, // 바이트의 인덱스 (0부터 시작)
pub is_code: F, // opcode이면 1, PUSH 데이터면 0
pub value: F, // 바이트 값
}
```

UnrolledBytecode 구조

```
pub struct UnrolledBytecode<F: Field> {
pub bytes: Vec<u8>, // 원본 바이트코드
pub rows: Vec<BytecodeRow<F>>, // 언롤링된 행들
}
```

언롤링 프로세스

함수: `unroll<F: Field>(bytes: Vec<u8>) -> UnrolledBytecode<F>`

언롤링은 다음 단계로 진행됩니다:

1. **헤더 행 생성**: 첫 번째 행은 항상 헤더로, 바이트코드의 길이를 저장합니다.

```
let mut rows = vec![BytecodeRow::<F> {
code_hash,
```

```
tag: F::from(BytecodeFieldTag::Header as u64), // tag = 0
index: F::zero(),
is_code: F::zero(),
value: F::from(bytes.len() as u64), // 길이 저장
}];
```

2. **바이트 순회**: 각 바이트를 순회하며 opcode인지 PUSH 데이터인지 판단합니다.

```
let mut push_rindex = 0;
for (index, byte) in bytes.iter().enumerate() {
// opcode인지 PUSH 데이터인지 판단
let is_code = push_rindex == 0;
// PUSH 명령어면 다음 N개 바이트는 데이터
push_rindex = if is_code {
get_push_size(*byte) // PUSH1~PUSH32면 1~32 반환
} else {
push_rindex - 1 // 데이터 카운트 감소
};

rows.push(BytecodeRow::<F> {
code_hash,
tag: F::from(BytecodeFieldTag::Byte as u64), // tag = 1
index: F::from(index as u64),
```

```
is_code: F::from(is_code as u64),
value: F::from(*byte as u64),
});
}
```

언롤링 예시

바이트코드 `[PUSH2, 0x12, 0x34, ADD]` 의 언롤링 결과:

Row	tag	index	is_code	value	설명
0	Header(0)	0	0	4	헤더: 길이 = 4
1	Byte(1)	0	1	PUSH2	opcode
2	Byte(1)	1	0	0x12	PUSH 데이터 1
3	Byte(1)	2	0	0x34	PUSH 데이터 2
4	Byte(1)	3	1	ADD	opcode

회로 구조

BytecodeCircuitConfig

회로의 주요 컬럼과 설정:

```
pub struct BytecodeCircuitConfig<F> {
// 고정 컬럼
q_enable: Column<Fixed>, // 활성화 플래그
q_first: Column<Fixed>, // 첫 행 표시
```

```
q_last: Column<Fixed>, // 마지막 행 표시

// Bytecode Table 컬럼

bytecode_table: BytecodeTable, // code_hash, tag, index, is_code, value, push_rl

// 보조 컬럼

push_data_left: Column<Advice>, // 남은 PUSH 데이터 개수

push_acc: Column<Advice>, // PUSH 데이터 RLC 누적값

value_rlc: Column<Advice>, // 바이트코드 RLC (Keccak 입력)

length: Column<Advice>, // 바이트코드 길이

push_data_size: Column<Advice>, // PUSH 명령어의 데이터 크기

// IsZero 가젯

push_data_left_is_zero: IsZeroConfig<F>, // push_data_left == 0 검증

index_length_diff_is_zero: IsZeroConfig<F>, // index + 1 == length 검증

// 룩업 테이블

push_table: [Column<Fixed>; 2], // (byte, push_size) 매핑

keccak_table: KeccakTable, // Keccak 해시 검증

}
```

주요 컬럼 설명

push_data_left

현재 행에서 남은 PUSH 데이터의 개수를 추적합니다.

- opcode 행: `push_data_left = 0`
- PUSH 명령어 직후: `push_data_left = push_data_size` (1~32)

- PUSH 데이터 행: `push_data_left = 이전 행의 push_data_left - 1`

예시 (PUSH2 0x12 0x34):

```
PUSH2: push_data_left = 0 (opcode)

0x12: push_data_left = 2 (첫 번째 데이터)

0x34: push_data_left = 1 (두 번째 데이터)
```

push_data_size

PUSH 명령어가 가져오는 데이터의 크기입니다.

- `PUSH1` : push_data_size = 1
- `PUSH2` : push_data_size = 2
- ...
- `PUSH32` : push_data_size = 32
- 기타 opcode: push_data_size = 0

이 값은 `push_table` 룩업을 통해 검증됩니다.

value_rlc

바이트코드의 Random Linear Combination으로, Keccak 해시 입력으로 사용됩니다.

```
// 헤더 다음 첫 바이트

value_rlc = value



// 이후 각 바이트

value_rlc = value_rlc_prev * challenge + value
```

push_acc와 push_rlc

PUSH 명령어의 데이터를 RLC로 누적하여 저장합니다.

- `push_acc` : PUSH 데이터를 읽는 동안 누적되는 중간값
- `push_rlc` : 완성된 PUSH 데이터의 RLC 값

동작 방식:

```
// opcode 행: push_acc = 0

// PUSH 데이터 행: push_acc = push_acc_prev * challenge + value

// 마지막 PUSH 데이터 행: push_rlc = push_acc (완성된 값)
```

PUSH 명령어 처리

PUSH 테이블 룩업

회로는 각 바이트 값에 대해 해당하는 PUSH 데이터 크기를 룩업합니다.

Push Table 구조:

```
byte (0~255) -> push_data_size

0x00~0x5F -> 0 (PUSH1 이전)

0x60 (PUSH1) -> 1

0x61 (PUSH2) -> 2

...

0x7F (PUSH32)-> 32

0x80~0xFF -> 0 (PUSH32 이후)
```

룩업 제약 조건:

```
meta.lookup_any("push_data_size_table_lookup", |meta| {
let  enable = is_byte(meta);
vec![
(enable.clone() * value, push_table[0]),
(enable * push_data_size, push_table[1]),
]
});
```

is_code 검증

바이트가 opcode인지 PUSH 데이터인지는 `push_data_left` 로 결정됩니다.

제약 조건:

```
// is_code == (push_data_left == 0)
cb.require_equal(
"cur.is_code == (cur.push_data_left == 0)",
is_code,
push_data_left_is_zero.is_zero_expression,
);
```

PUSH 데이터 전환

Byte to Byte 전환 시:

```
// 다음 행의 push_data_left 계산
next.push_data_left = if  cur.is_code {
cur.push_data_size // opcode면 새로운 PUSH 시작
} else {
cur.push_data_left - 1  // 데이터면 카운트 감소
}
```

PUSH RLC 누적

PUSH 데이터의 RLC는 다음과 같이 계산됩니다:

```
// opcode 행: push_acc = 0
if  is_code {
push_acc = 0;
}

// PUSH 데이터 행: 누적
if !is_code_next {
push_acc_next = push_acc * challenge + value_next;
}

// 마지막 PUSH 데이터 행: push_rlc에 저장
if  is_code_next { // 다음이 opcode면 현재가 마지막 데이터
push_rlc = push_acc;
```

```
}
```

Code Hash 검증

Keccak 테이블 룩업

바이트코드의 마지막 행(Byte to Header 전환)에서 Keccak 테이블 룩업을 수행합니다.

제약 조건 (Keccak256 모드):

```
meta.lookup_any("keccak256_table_lookup", |meta| {
let  enable = is_byte_to_header(meta);
vec![
(enable.clone(), keccak_table.is_final),
(enable.clone() * value_rlc, keccak_table.input_rlc),
(enable.clone() * length, keccak_table.input_len),
(enable * code_hash, keccak_table.output_rlc),
]
});
```

이 룩업은 다음을 검증합니다:

- `value_rlc` : 바이트코드의 RLC (입력)
- `length` : 바이트코드 길이
- `code_hash` : 계산된 Keccak256 해시 (출력)

Poseidon 해시 (선택적)

`poseidon-codehash` 기능이 활성화된 경우, Keccak 대신 Poseidon 해시를 사용합니다.

```
#[cfg(feature = "poseidon-codehash")]
pub type CircuitConfig<F> = ToHashBlockCircuitConfig<F, HASHBLOCK_BYTES_IN_FIEL
```

빈 코드 해시

길이가 0인 바이트코드는 미리 정의된 빈 해시 값을 사용합니다.

```
let empty_hash = if cfg!(feature = "poseidon-codehash") {
POSEIDON_CODE_HASH_EMPTY
} else {
EMPTY_CODE_HASH_LE // Keccak256("")
};
```

행 전환 규칙

Header to Header

빈 바이트코드 (길이 = 0):

```
// cur.length == 0
// cur.hash == EMPTY_HASH
```

Header to Byte

바이트코드 시작:

```
// next.length == cur.length

// next.index == 0

// next.is_code == 1 (첫 바이트는 항상 opcode)

// next.hash == cur.hash

// next.value_rlc == next.value (RLC 초기화)
```

Byte to Byte

바이트코드 내부:

```
// next.length == cur.length

// next.index == cur.index + 1

// next.hash == cur.hash

// next.value_rlc == cur.value_rlc * challenge + next.value

// next.push_data_left == cur.is_code ? cur.push_data_size : cur.push_data_left
```

Byte to Header

바이트코드 종료:

```
// cur.index + 1 == cur.length (마지막 바이트)

// keccak_table_lookup(cur.value_rlc, cur.length, cur.hash)

// push_rlc == push_acc (마지막 PUSH 데이터 완성)
```

패딩 처리

회로 크기가 실제 바이트코드보다 큰 경우, 나머지 행은 패딩으로 채워집니다.

패딩 행 구조:

```
BytecodeRow {
code_hash: EMPTY_HASH,
tag: Header,
index: 0,
is_code: 0,
value: 0, // length = 0
}
```

패딩 행은 빈 바이트코드의 헤더와 동일한 형태입니다.

제약 조건 요약

1. Tag 제약

```
// tag는 boolean (0 또는 1)
tag * (1 - tag) == 0
```

2. Header 제약

```
// Header 행: index == 0, value == length
```

```
if  tag == Header {
index == 0
value == length
}
```

3. Byte 제약

```
// Byte 행: is_code 검증, PUSH 테이블 룩업
if  tag == Byte {
is_code == (push_data_left == 0)
push_table_lookup(value, push_data_size)
if  is_code {
push_acc == 0
}
}
```

4. 전환 제약

```
// Header to Byte
if  cur.tag == Header && next.tag == Byte {
next.length == cur.length
next.index == 0
next.is_code == 1
```

```
next.hash == cur.hash

next.value_rlc == next.value

}


// Byte to Byte

if  cur.tag == Byte && next.tag == Byte {

next.length == cur.length

next.index == cur.index + 1

next.hash == cur.hash

next.value_rlc == cur.value_rlc * challenge + next.value

next.push_data_left == cur.is_code ? cur.push_data_size : cur.push_data_left - 1

}


// Byte to Header

if  cur.tag == Byte && next.tag == Header {

cur.index + 1 == cur.length

keccak_lookup(cur.value_rlc, cur.length, cur.hash)

push_rlc == push_acc

}
```

5. 빈 코드 제약

```
// Header to Header 또는 마지막 행
```

```
if (cur.tag == Header && next.tag == Header) || q_last {
cur.length == 0
cur.hash == EMPTY_HASH
}
```

Witness 생성

할당 프로세스

함수: `assign_bytecode()`

1. **초기화**: 각 바이트코드에 대해 변수 초기화

```
let mut push_data_left = 0;
let mut push_data_size = 0;
let mut push_acc = Value::known(F::zero());
let mut push_rlc = Value::known(F::zero());
let mut value_rlc = challenges.keccak_input().map(|_| F::zero());
```

2. **헤더 행 할당**: 코드 해시와 길이 계산

```
let code_hash = challenges.evm_word().map(|challenge| {
rlc::value(&bytecode.rows[0].code_hash.to_le_bytes(), challenge)
});
```

3. **바이트 행 할당**: 각 바이트에 대해

```
for (idx, row) in  bytecode.rows.iter().enumerate() {
if  idx > 0 { // 헤더 제외
let  is_code = push_data_left == 0;
push_data_size = get_push_size(row.value);
// PUSH RLC 계산
if  is_code {
let  push_accumulator = make_push_rlc(
challenges.evm_word(),
&bytecode.rows[idx+1..idx+1+push_data_size]
);
push_rlc = push_accumulator.0;
push_acc_iter = push_accumulator.1.into_iter();
}
// value_rlc 누적
value_rlc = value_rlc * challenge + row.value;
}
// 행 할당
set_row(region, offset, ...);
}
```

PUSH RLC 계산

```
fn  make_push_rlc(rand: Value<F>, rows: &[BytecodeRow<F>])
```

```
-> (Value<F>, Vec<Value<F>>)
{
let mut acc = Value::known(F::zero());
let intermediates = rows.iter().map(|row| {
acc = acc * rand + Value::known(row.value);
acc
}).collect();
(acc, intermediates)
}
```

이 함수는:

- 최종 RLC 값 반환
- 중간 누적값들의 벡터 반환 (각 PUSH 데이터 행에 할당)

테이블 연동

Bytecode Table

다른 회로(특히 EVM Circuit)는 Bytecode Table을 통해 바이트코드 정보를 조회합니다.

테이블 스키마:

```
pub struct BytecodeTable {
pub code_hash: Column<Advice>, // 코드 해시
pub tag: Column<Advice>, // Header/Byte
pub index: Column<Advice>, // 바이트 인덱스
```

```
pub  is_code: Column<Advice>, // opcode 여부
pub  value: Column<Advice>, // 바이트 값
pub  push_rlc: Column<Advice>, // PUSH 데이터 RLC
}
```

사용 예시 (EVM Circuit에서):

```
// 특정 코드 해시의 특정 인덱스 바이트 조회
lookup(
code_hash,
Byte, // tag
index,
is_code,
value,
push_rlc
)
```

Keccak Table

Bytecode Circuit은 Keccak Table을 사용하여 코드 해시를 검증합니다.

룩업 시점: 각 바이트코드의 마지막 바이트 (Byte to Header 전환)

룩업 데이터:

- 입력: `value_rlc` (바이트코드 RLC), `length`
- 출력: `code_hash`

최적화 기법

1. RLC를 통한 데이터 압축

바이트코드 전체를 하나의 필드 원소로 압축:

```
value_rlc = b[0] + b[1]*r + b[2]*r² + ... + b[n-1]*r^(n-1)
```

여기서 `r` 은 challenge 값입니다.

2. PUSH 데이터 미리 계산

PUSH 명령어를 만나면 즉시 모든 PUSH 데이터의 RLC를 계산하여 `push_rlc` 에 저장합니다. 이를 통해:

- EVM Circuit이 PUSH 데이터를 효율적으로 검증 가능
- 중간 계산 없이 최종 값 직접 사용

3. IsZero 가젯 재사용

`push_data_left == 0` 과 `index + 1 == length` 검증에 IsZero 가젯을 재사용하여 회로 크기 최적화

4. 고정 테이블 사용

PUSH 테이블은 고정 컬럼으로 구현되어 모든 인스턴스에서 재사용 가능

구현 예시

간단한 바이트코드 검증

```
// 바이트코드: PUSH1 0x42 ADD
```

```rust
let bytes = vec![0x60, 0x42, 0x01];
let unrolled = unroll::<F>(bytes);

// 언롤링 결과:
// Row 0: Header, index=0, value=3 (length)
// Row 1: Byte, index=0, is_code=1, value=0x60 (PUSH1)
// Row 2: Byte, index=1, is_code=0, value=0x42 (data)
// Row 3: Byte, index=2, is_code=1, value=0x01 (ADD)
```

중첩 PUSH 처리

```rust
// PUSH2 PUSH1 (잘못된 코드이지만 구조적으로 유효)
let bytes = vec![0x61, 0x60, 0x42];

// Row 1: PUSH2, is_code=1, push_data_left=0, push_data_size=2
// Row 2: 0x60, is_code=0, push_data_left=2 (PUSH2의 첫 데이터)
// Row 3: 0x42, is_code=0, push_data_left=1 (PUSH2의 둘째 데이터)
```

관련 문서

이전/다음

- **이전**: State Circuit - 상태 관리 회로
- **다음**: Keccak Circuit - 해시 함수 회로

같은 섹션 (핵심 회로)

- EVM Circuit - Bytecode Table 사용
- State Circuit - 상태 관리
- Keccak Circuit - Code hash 계산
- Tx Circuit - 트랜잭션 검증
- Copy Circuit - 복사 연산

관련 테이블

- Bytecode Table - 테이블 스키마 상세
- Keccak Table - Keccak 해시 결과
- 테이블 시스템 - 테이블 개요

관련 시스템

- Bus-Mapping 개요 - 바이트코드 수집
- Witness 생성 - Witness 데이터
- 데이터 흐름 - 전체 데이터 흐름

구현 세부사항

- Halo2 통합 - Halo2 API
- Lookup 메커니즘 - 룩업 동작
- RLC 기법 - Random Linear Combination

참고 자료

주요 파일:

- `zkevm-circuits/src/bytecode_circuit/circuit.rs` : 회로 구현
- `zkevm-circuits/src/bytecode_circuit/bytecode_unroller.rs` : 언롤링 로직
- `zkevm-circuits/src/table.rs` : BytecodeTable 정의
- `zkevm-circuits/src/util.rs` : `get_push_size()` 함수

핵심 함수:

- `unroll()` : 바이트코드 언롤링
- `assign_bytecode()` : Witness 할당
- `make_push_rlc()` : PUSH 데이터 RLC 계산
- `get_push_size()` : PUSH 명령어 크기 반환